

Nivel 1

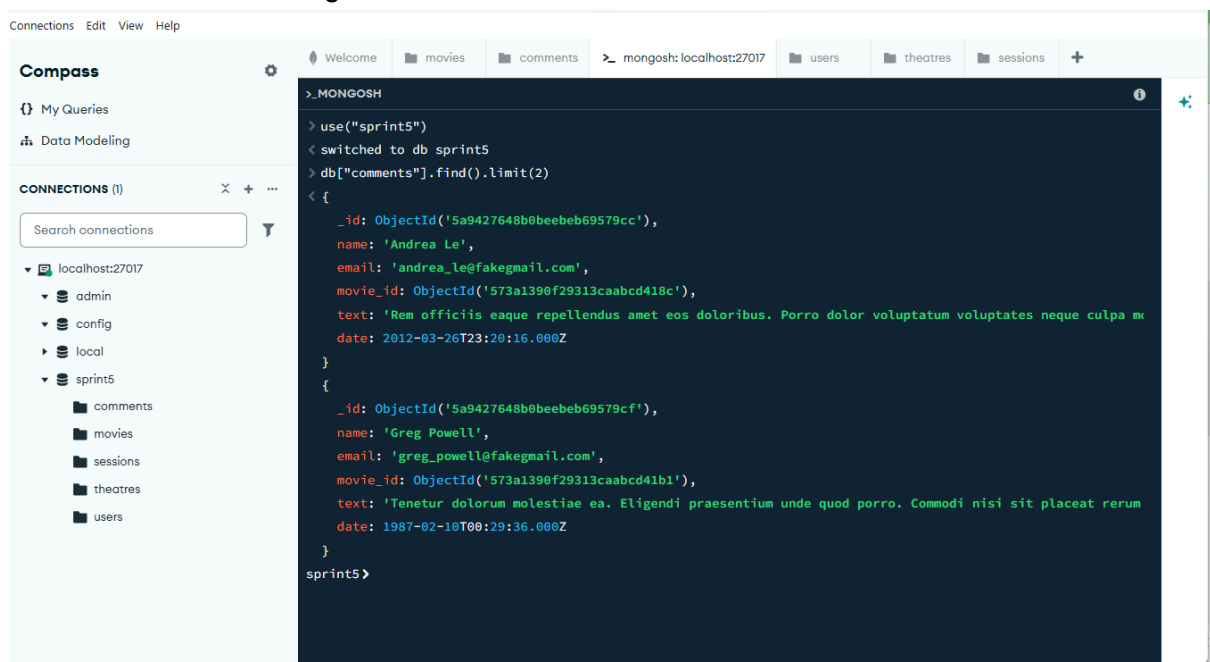
Nivel 1

Crea una base de datos con MongoDB utilizando como colecciones los archivos adjuntos.

Ejercicio 1

- Muestra los 2 primeros comentarios que aparecen en la base de datos.
- ¿Cuántos usuarios tenemos registrados?
- ¿Cuántos cines existen en el estado de California?
- ¿Cuál fue el primer usuario en registrarse?
- ¿Cuántas películas de comedia existen en nuestra base de datos?

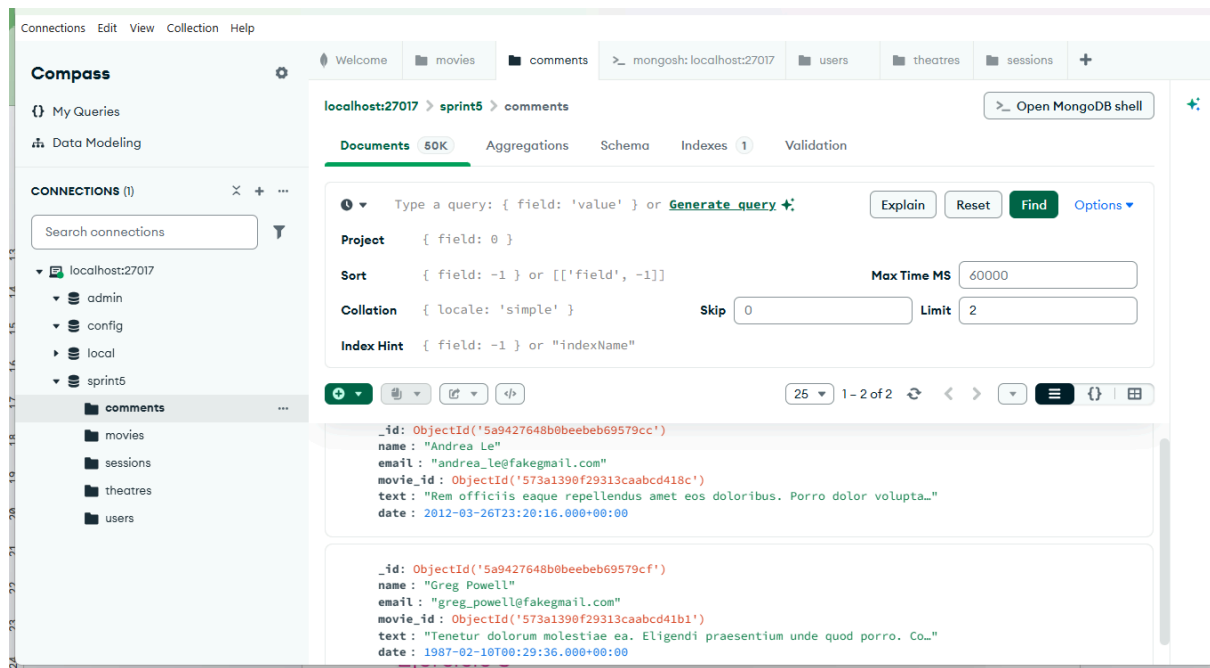
Captura de los dos primeros comentarios:
Mediante la terminal integrada:



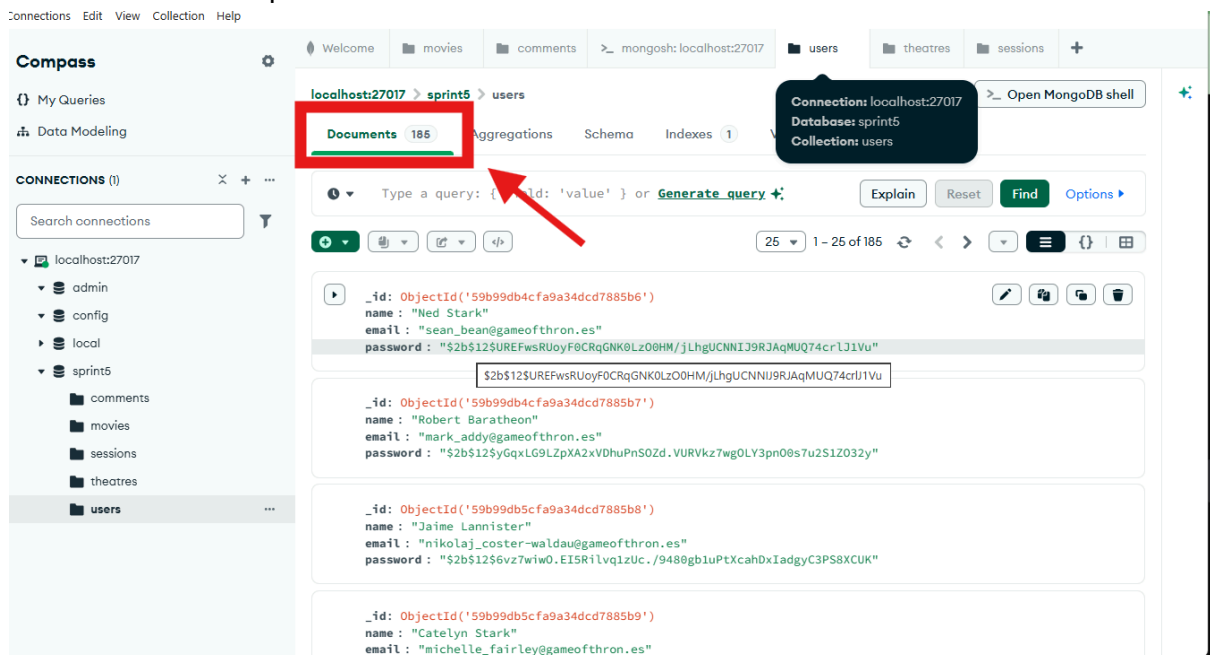
The screenshot shows the MongoDB Compass interface. On the left, the 'Connections' panel shows a connection to 'localhost:27017' with a tree view of databases and collections. The 'sprint5' database is selected, showing collections: 'comments', 'movies', 'sessions', 'theatres', and 'users'. The main panel displays a terminal window with the following commands and output:

```
> use("sprint5")
< switched to db sprint5
> db["comments"].find().limit(2)
< {
  _id: ObjectId('5a9427648b0beebe69579cc'),
  name: 'Andrea Le',
  email: 'andrea_le@fakegmail.com',
  movie_id: ObjectId('573a1390f29313caabcd418c'),
  text: 'Rem officiiis eaque repellendus amet eos doloribus. Porro dolor voluptatum voluptates neque culpa m',
  date: 2012-03-26T23:20:16.000Z
}
{
  _id: ObjectId('5a9427648b0beebe69579cf'),
  name: 'Greg Powell',
  email: 'greg_powell@fakegmail.com',
  movie_id: ObjectId('573a1390f29313caabcd41b1'),
  text: 'Tenetur dolorum molestiae ea. Eligendi praesentium unde quod porro. Commodi nisi sit placeat rerum',
  date: 1987-02-10T00:29:36.000Z
}
sprint5>
```

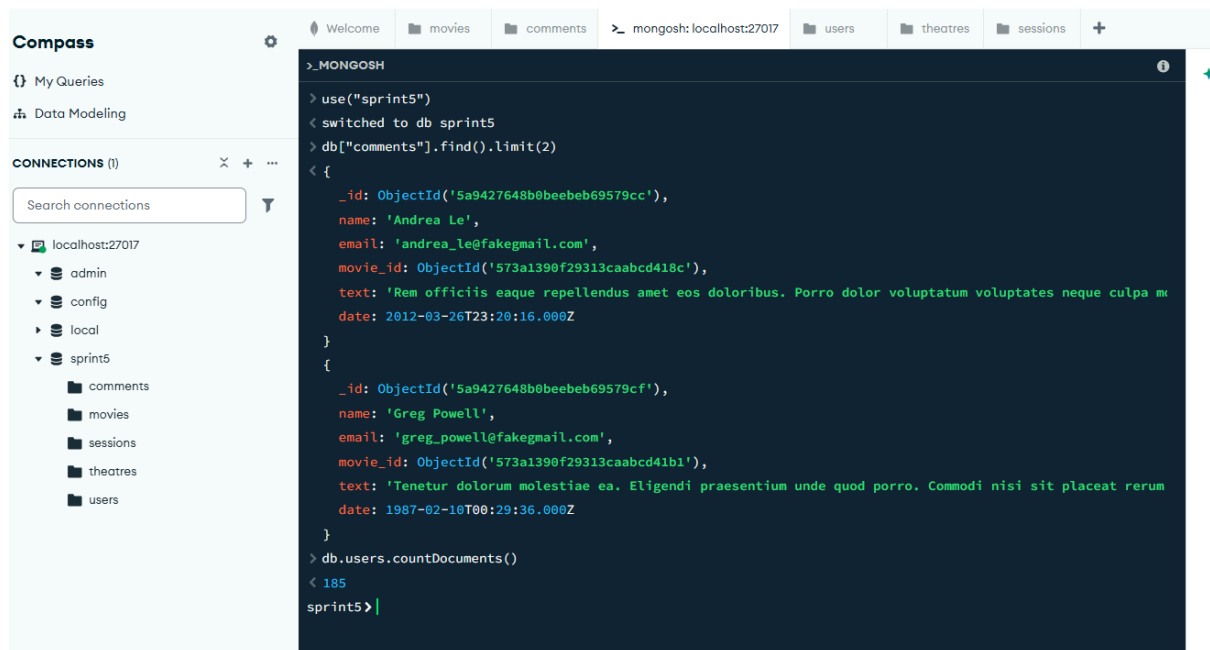
Usando la Interfaz Visual de Compass:



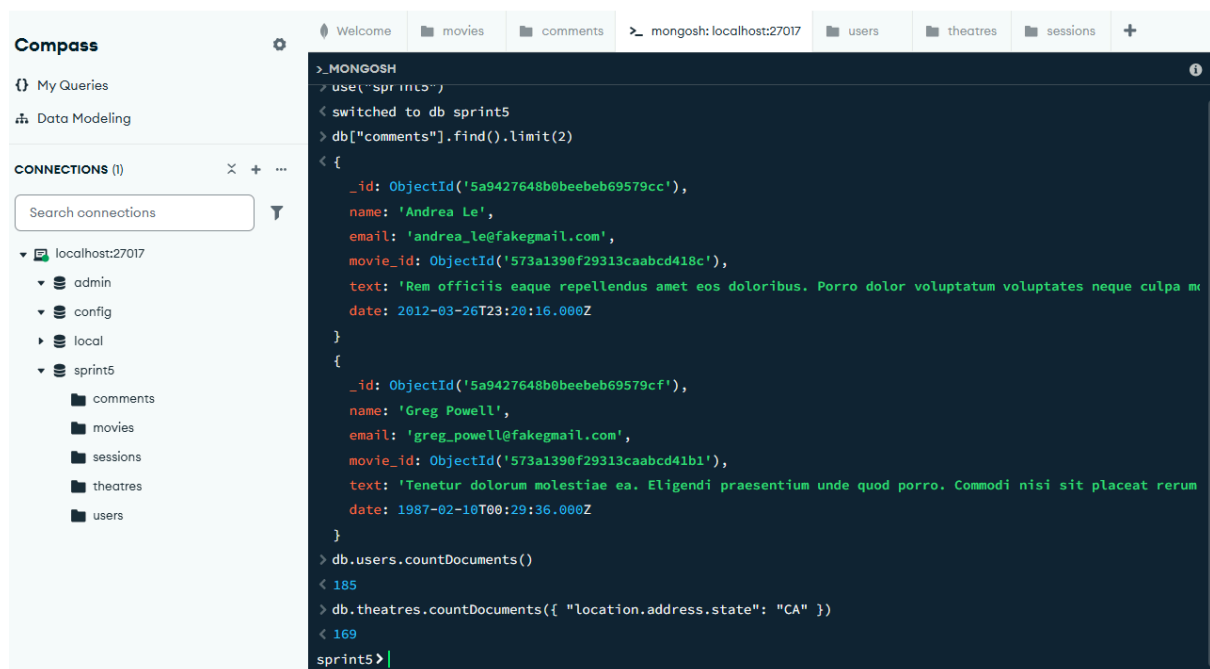
Captura de usuarios registrados:
usando interfaz compass:



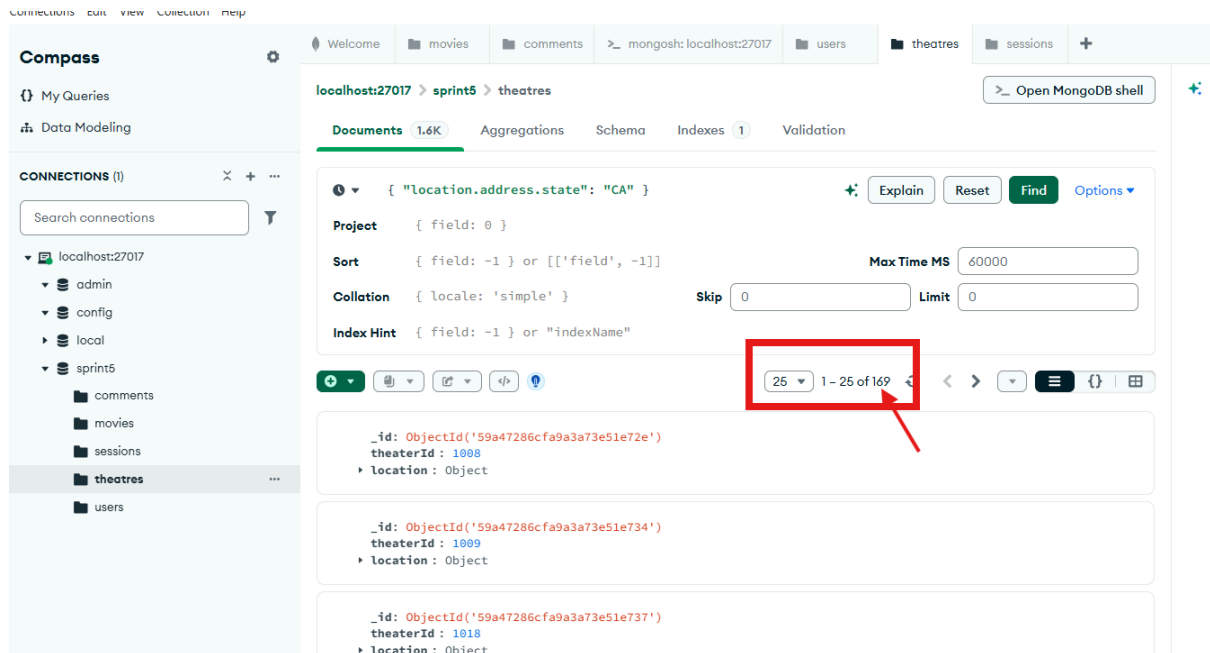
usando terminal:



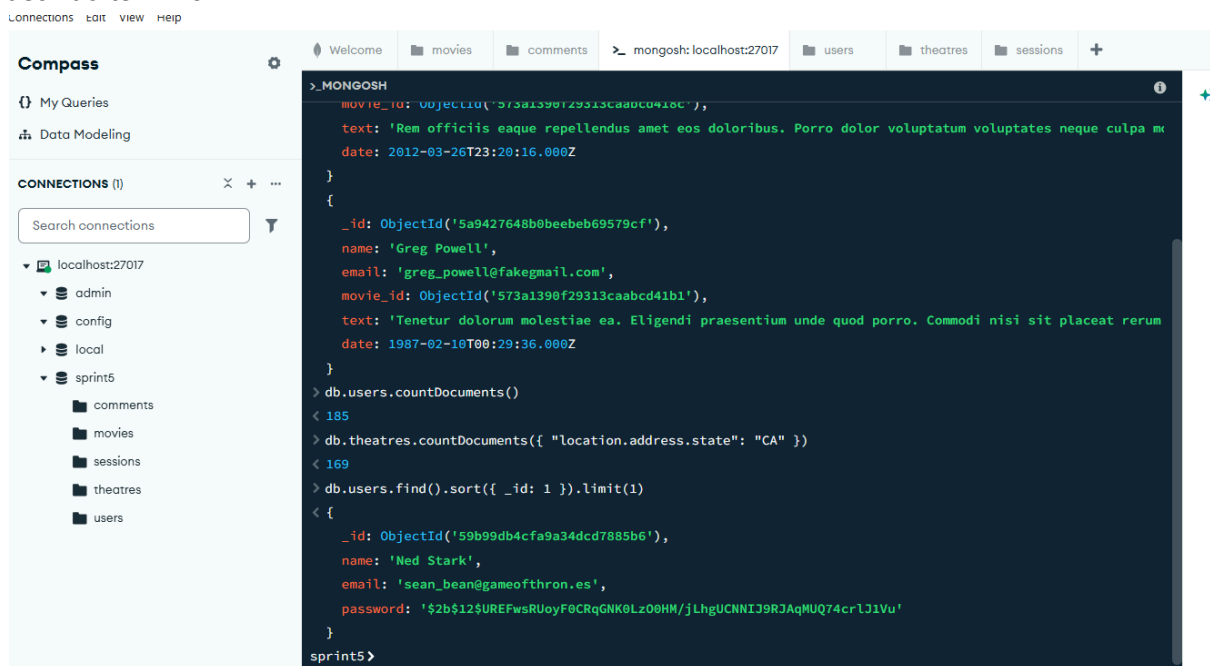
Cuantos cines en el estado de california:
usando terminal:



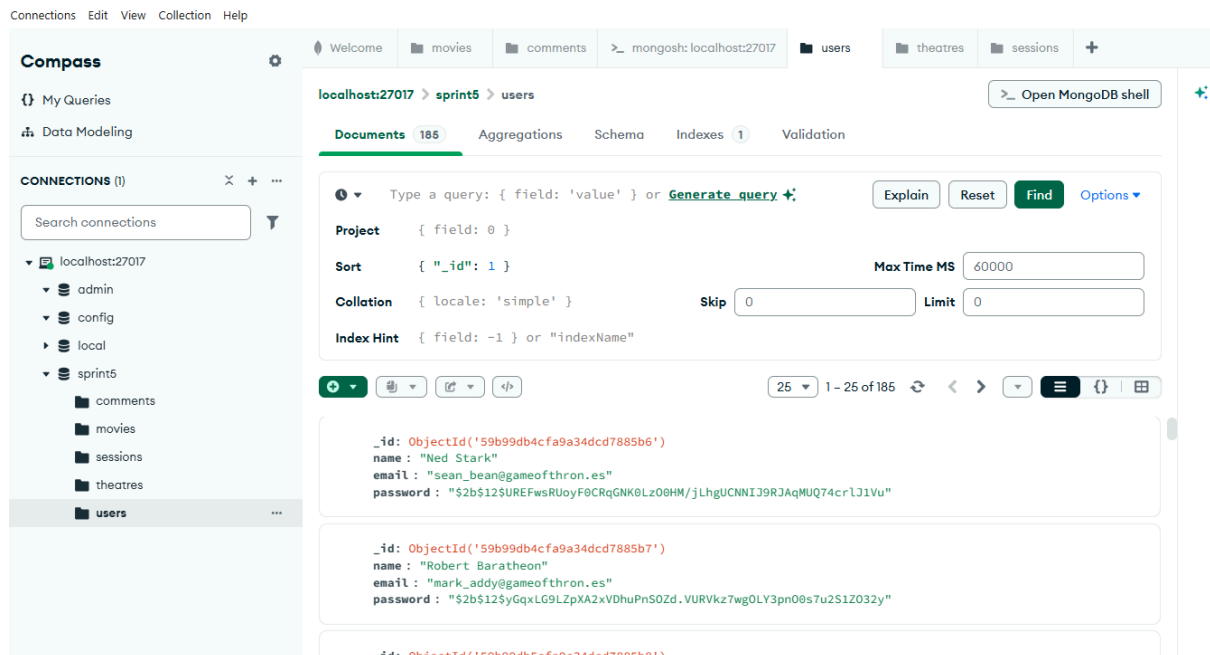
usando compass:



Primer usuario en registrarse:
usando terminal:



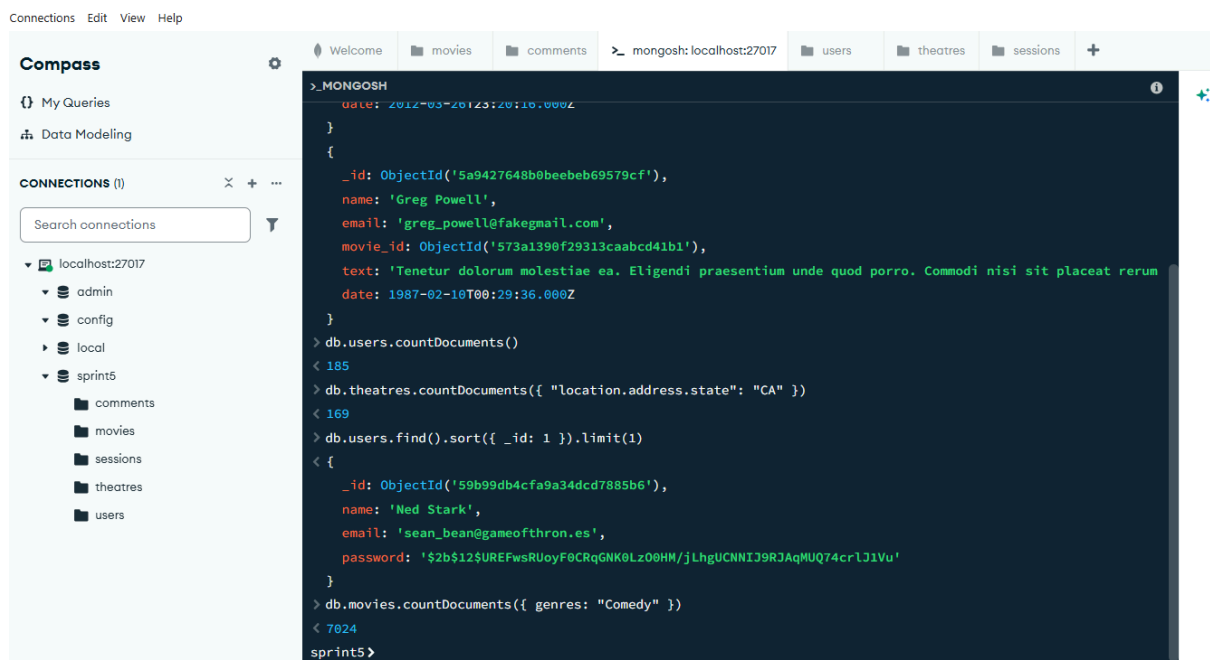
usando compass:



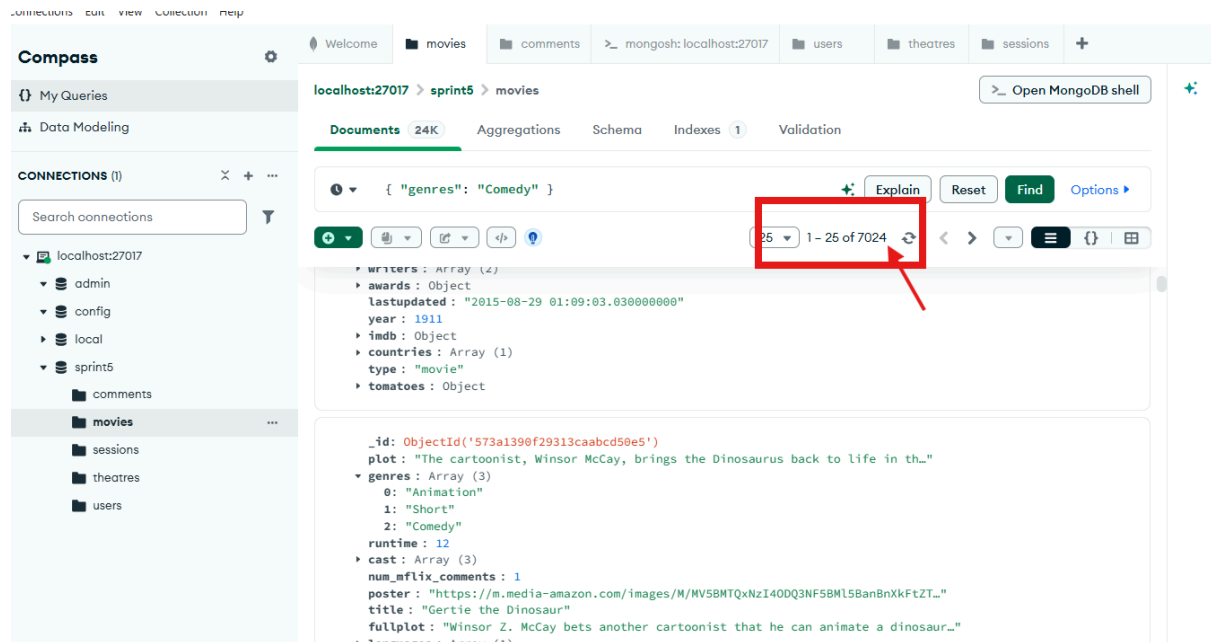
En MongoDB, cada registro recibe automáticamente un identificador único denominado ObjectId. A diferencia de los ID numéricos tradicionales, este código incluye grabada en sus primeros caracteres la fecha y hora exacta de creación del registro.

Gracias a esta propiedad, al ordenar la colección de usuarios por su campo `_id` de menor a mayor (`{ _id: 1 }`), los datos quedan organizados automáticamente en orden cronológico. De esta forma, el primer registro de la lista corresponde con total precisión al primer usuario registrado en la plataforma, sin necesidad de contar con un campo adicional de fecha de registro.

Películas de Comedia:
usando terminal:



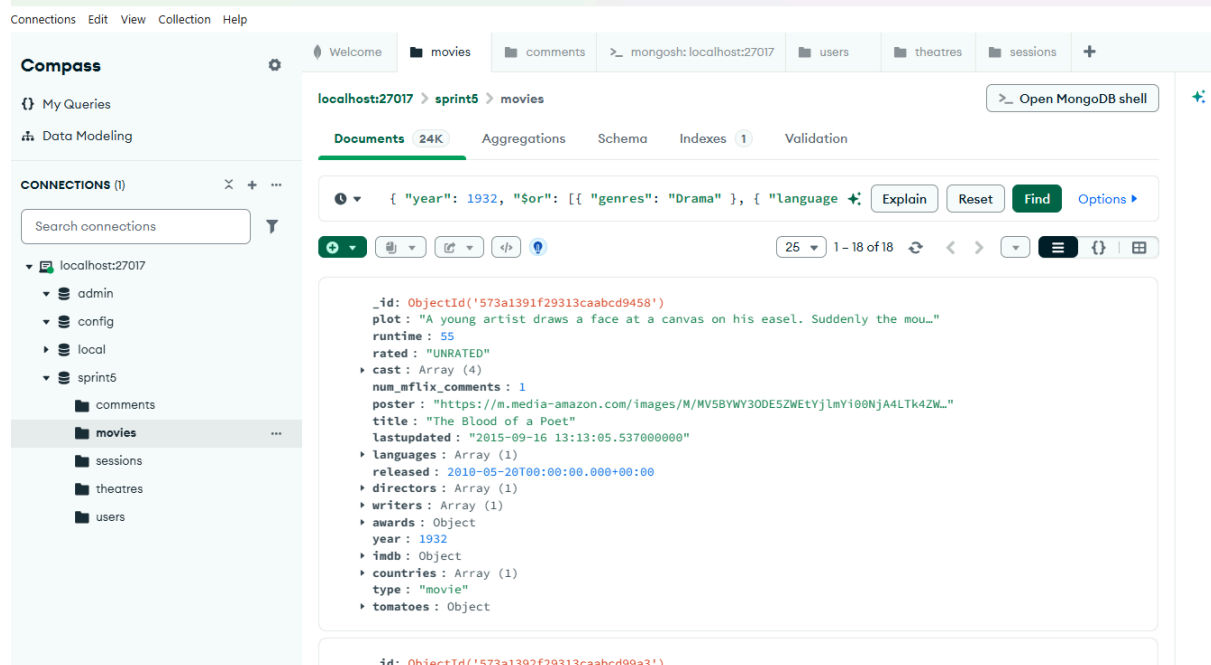
usando compass:



Ejercicio 2

Muéstrame todos los documentos de las películas producidas en 1932, pero que el género sea drama o estén en francés.

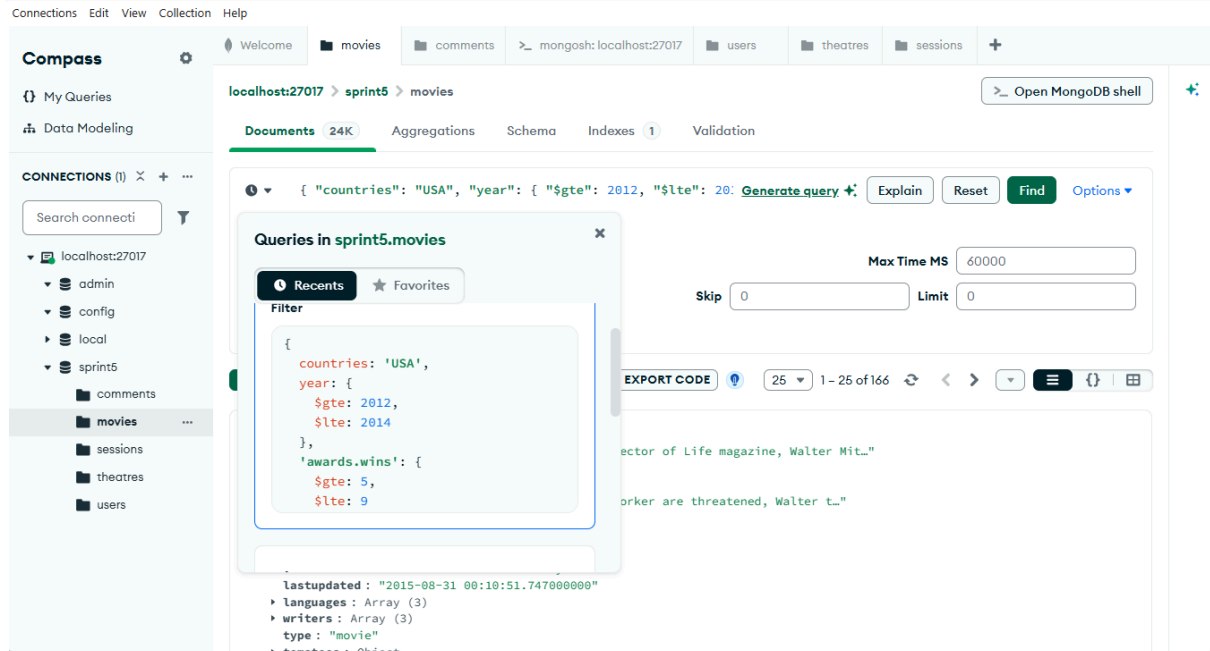
Captura resultado consulta:



\$or: Permite evaluar dos condiciones donde basta con que se cumpla al menos una de las dos, en este caso genero drama y lenguaje frances.

Ejercicio 3

Muéstrame todos los documentos de películas estadounidenses que tengan entre 5 y 9 premios que fueron producidas entre 2012 y 2014.



\$gte (Greater Than or Equal)

\$lte (Less Than or Equal)

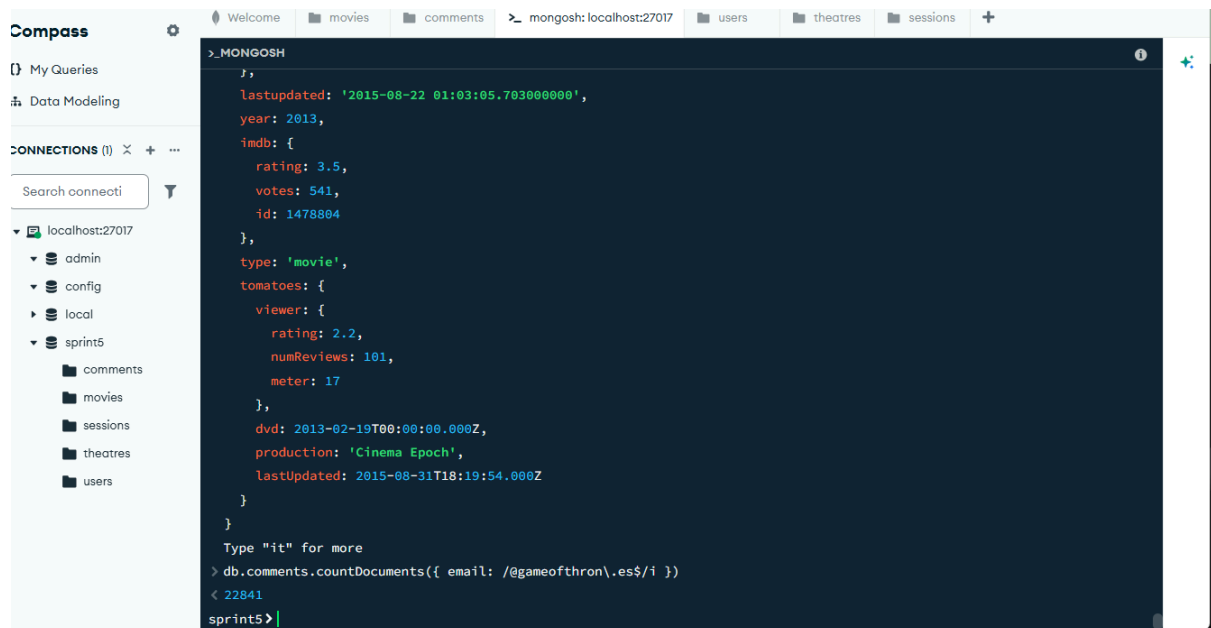
Nivel 2

Nivel 2

Ejercicio 1

Cuenta cuántos comentarios escribe un usuario que utiliza "GAMEOFTHRON.ES" como dominio de correo electrónico.

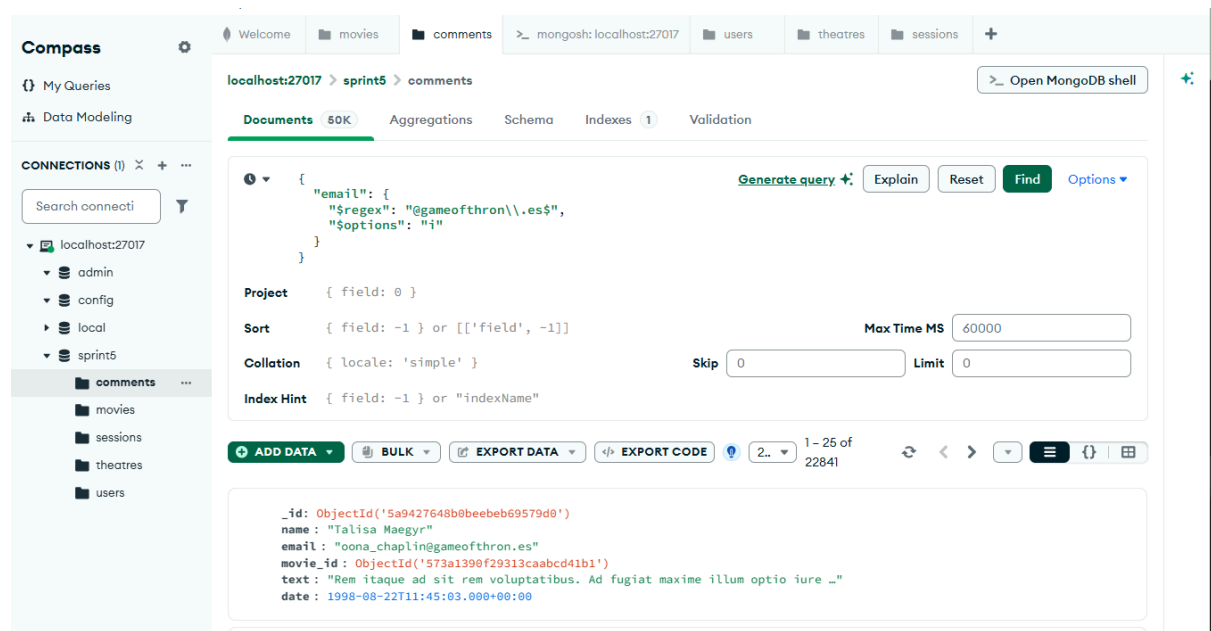
Consulta usando la terminal:



```
>_MONGOSH
{
  lastupdated: '2015-08-22 01:03:05.703000000',
  year: 2013,
  imdb: {
    rating: 3.5,
    votes: 541,
    id: 1478804
  },
  type: 'movie',
  tomatoes: {
    viewer: {
      rating: 2.2,
      numReviews: 101,
      meter: 17
    },
    dvd: 2013-02-19T00:00:00.000Z,
    production: 'Cinema Epoch',
    lastUpdated: 2015-08-31T18:19:54.000Z
  }
}

Type "it" for more
> db.comments.countDocuments({ email: /@gameofthron\.es$/i })
< 22841
sprint5>
```

usando compass:



\$regex: Permite evaluar patrones de texto flexibles dentro de la base de datos en lugar de requerir coincidencias exactas.

@gameofthron\\.es\$:

\\.es: garantiza que el punto se interprete como un signo ortográfico literal y no como el comodín universal de sintaxis regex.

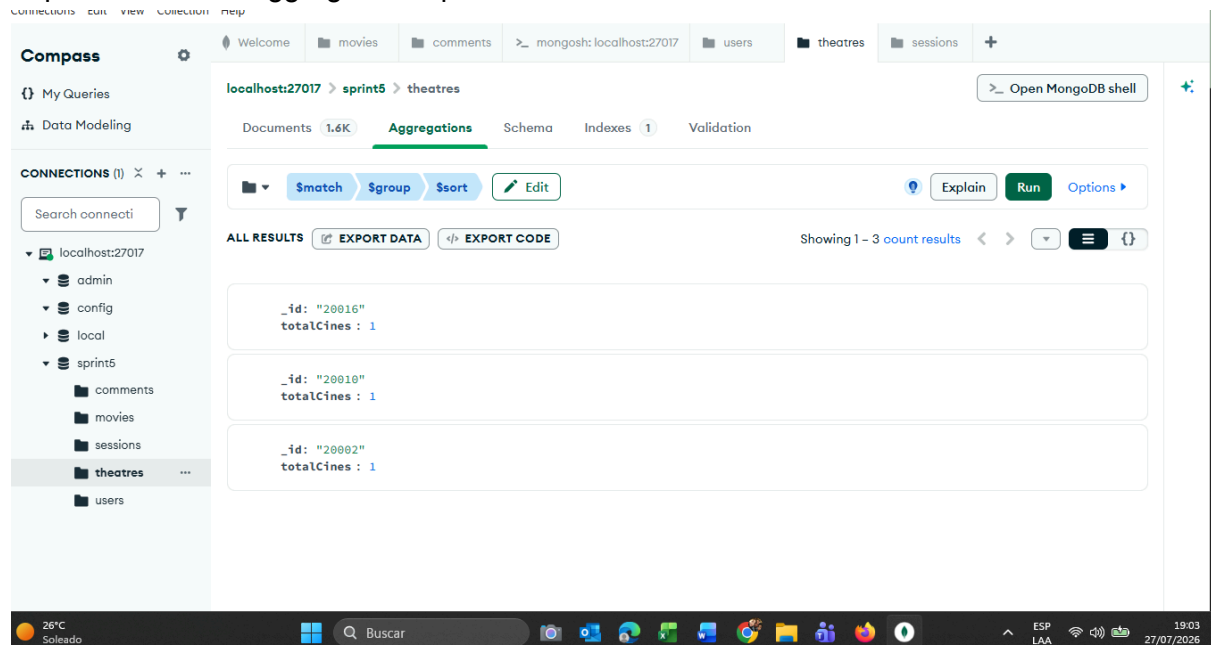
\$ al final ancla la búsqueda exclusivamente al término de la cadena de texto, omitiendo variaciones secundarias no deseadas (por ejemplo, .es.com).

"\$options": "i"): Aplica la propiedad case-insensitivity (insensibilidad a mayúsculas y minúsculas), asegurando la captura de registros sin importar cómo hayan sido introducidos en la base de datos (p. ej., @gameofthron.es o @GAMEOFTHRON.ES).

Ejercicio 2

¿Cuántos cines existen en cada código postal situados dentro del estado Washington DC (DC)?

Captura resultado Aggregation Pipeline:



En MongoDB, un pipeline funciona como una cinta transportadora de fábrica: los datos entran por un extremo y van pasando por distintas etapas (stages) en secuencia. La salida de una etapa se convierte automáticamente en la entrada de la siguiente, en el presente ejercicio se realizó lo siguiente:

Etapas 1: \$match (Filtro)

```
{ "$match": { "location.address.state": "DC" } }
```

Equivalencia a un WHERE "location.address.state = 'DC'".

De todos los cines que hay en la base de datos, esta primera etapa descarta los que no nos interesan y se queda únicamente con aquellos cuya ubicación dentro del documento (location.address.state) sea igual a "DC" (Washington DC), el resultado que pasa a la siguiente etapa: Solo los cines de Washington DC.

Etapa 2: \$group (Agrupamiento y Conteo)

```
{ "$group": { "_id": "$location.address.zipcode", "totalCines": { "$sum": 1 } } }
```

Equivale a GROUP BY location.address.zipcode sumando un COUNT(*).

Toma los cines de Washington DC de la etapa anterior y los junta en "cajas" según su código postal.

`_id: "$location.address.zipcode"`: Le indica a MongoDB cuál será el criterio para agrupar. El signo \$ al principio de \$location... significa "usa el valor del campo zipcode que hay dentro del documento".

`totalCines: { "$sum": 1 }`: Crea un nuevo campo llamado totalCines. Por cada documento de cine que cae dentro de un mismo código postal, le suma 1 al contador.

Etapa 3: \$sort (Orden)

```
{ "$sort": { "totalCines": -1 } }
```

Equivale a ORDER BY totalCines DESC.

Toma el resultado agrupado de la etapa 2 y lo ordena según el campo que acabamos de calcular (totalCines).

El valor -1 indica orden descendente (de mayor a menor). Si usáramos 1, sería ascendente.

Sirve para colocar en primer lugar los códigos postales que tengan más cines registrados.

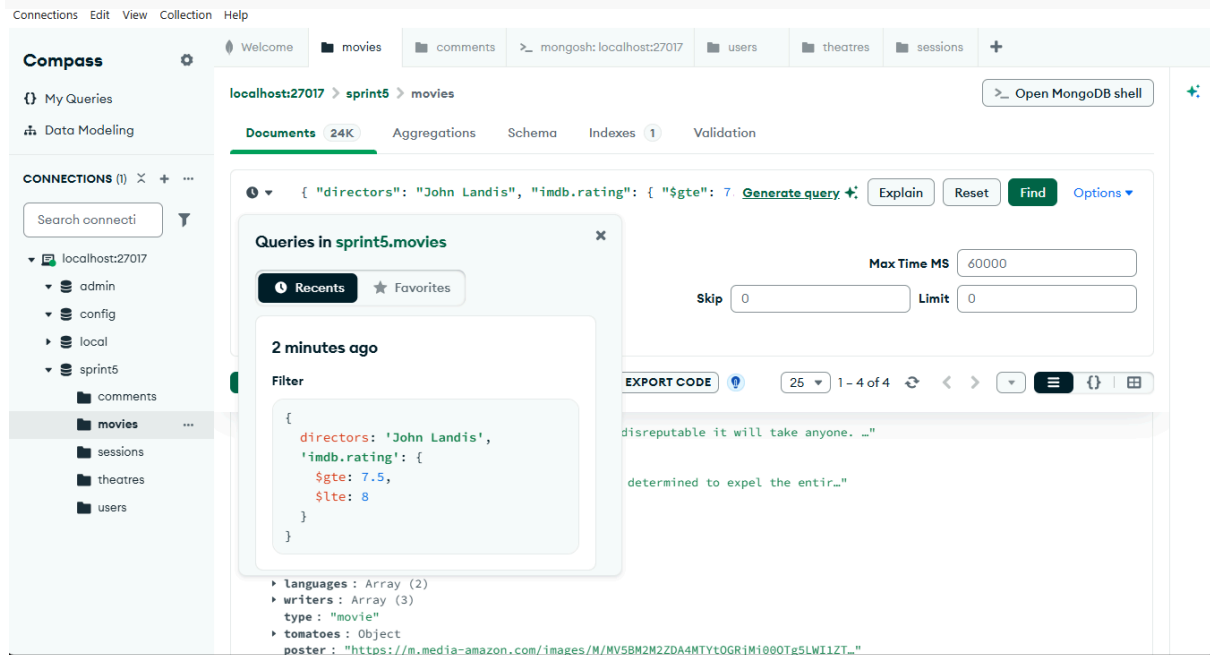
Nivel 3

Nivel 3

Ejercicio 1

Encuentra todas las películas dirigidas por John Landis con una puntuación IMDb (Internet Movie Database) de entre 7,5 y 8.

Captura resultado películas



directors: "John Landis": Busca las películas donde el director "John Landis" aparezca dentro de la lista/array del campo directors.

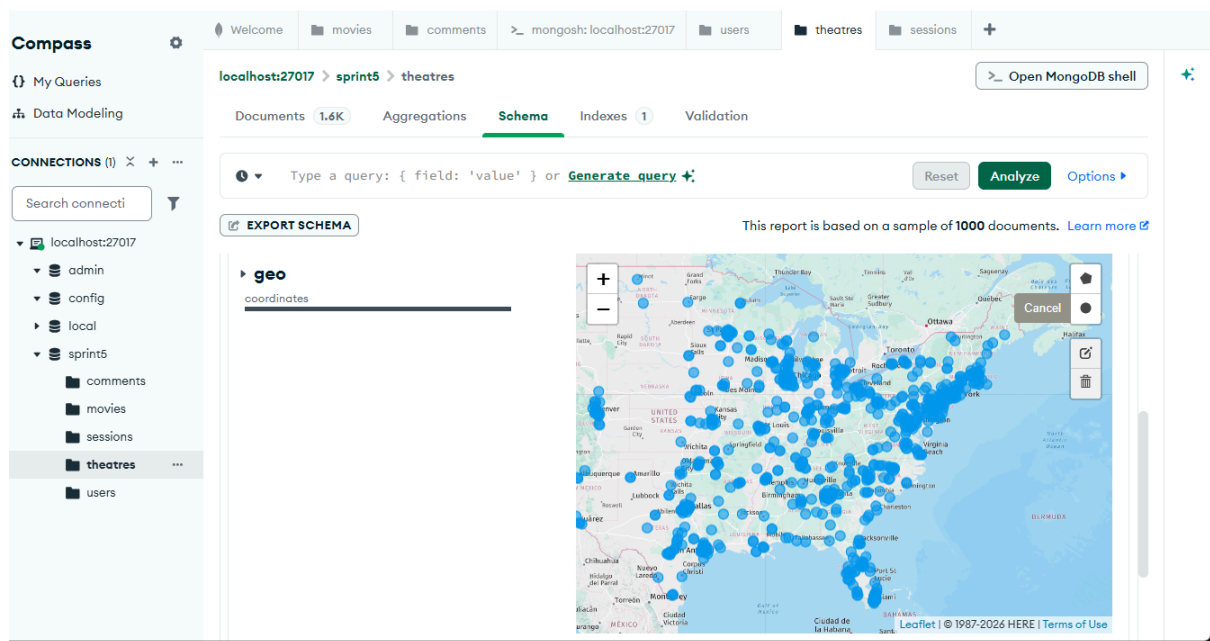
"imdb.rating": { \$gte: 7.5, \$lte: 8 }: Accede a la puntuación de la película a través del subcampo anidado rating dentro del objeto imdb:

- \$gte: 7.5 Selecciona valores mayores o iguales a 7.5.
- \$lte: 8 Selecciona valores menores o iguales a 8.0.

Ejercicio 2

Muestra en un mapa la ubicación de todos los teatros de la base de datos.

Captura de ubicaciones desde el compass:



Captura desde google maps:

